

LumaLife 综合生活助手平台

部署文档

Deployment Guide

文档编号	LUMALIFE-DEP-003
版本	2.0 · 当前微服务验收版
更新日期	2026-09-03
状态	最终答辩前验收基线

文档编号：LUMALIFE-DEP-003 版本：3.0（最终答辩前验收版） 更新日期：2026-09-03 状态：PARTIAL。本文描述当前代码与配置中的部署方式；本次文档更新未启动容器、集群或服务器，运行结果沿用已注明的本地或既有证据。

1 文档目的与部署边界

本文说明 LumaLife 当前微服务版本的本地开发、Docker Compose、Kubernetes/Kustomize、CI/CD 和阿里云 ECS K3s 部署方式，并给出配置、验证、排障和回滚边界。

当前默认调用链为：

```
frontend -> backend BFF -> identity / merchant / order / assistant
order <-> merchant via RabbitMQ Saga
```

关键边界：

- backend 是统一 BFF/API facade，不计入业务微服务；
- 业务微服务共 4 个：identity、merchant、order、assistant；
- Compose 默认 8 个服务：6 个应用、MySQL、RabbitMQ；
- 默认 Compose 使用一个 MySQL 实例中的 3 个逻辑服务数据库；
- Kubernetes 使用 3 个服务数据库实例，并保留 legacy MySQL 供迁移/回滚；
- `monolith` profile 仍存在，不得描述为已完全删除单体。

2 组件、端口与数据

2.1 应用与基础设施

组件	类型	主机端口	职责
frontend	应用	5173	React 静态页面与反向代理
backend	应用/BFF	8080	52 个公开 API、鉴权与编排
identity-service	业务微服务	8081	登录、Token、资料、地址
merchant-service	业务微服务	8082	商家、目录、收藏、客服、库存
order-service	业务微服务	8083	购物车、订单、支付、券码、评价
assistant-service	业务微服务	8084	AI provider 边界与确定性降级
mysql	基础设施	3306	3 个服务数据库及 legacy 数据库
rabbitmq	基础设施	不默认映射	Outbox/Inbox/Saga 消息传递

RabbitMQ 容器内部使用 5672/15672。Kubernetes Service 使用集群内端口，frontend Service 为 80，其他应用端口与上表一致。

2.2 数据库归属

数据库	所有者	主要数据
life_assistant_identity	identity-service	账号、地址、会话
life_assistant_merchant	merchant-service	商家、目录、收藏、客服、库存与消息表
life_assistant_order	order-service	购物车、订单、支付、券码、评价与 Saga 表
life_assistant	legacy	单体基线、迁移与回滚，不是默认业务事实源

assistant-service 无业务数据库。版本迁移为 V001-V019，迁移脚本对已应用版本校验 SHA-256。

3 环境准备

3.1 软件要求

工具	项目要求	用途
Git	支持当前仓库	获取源码与记录部署提交
JDK	17+	backend 与 services 构建/运行
Maven	3.9+	Java 构建
Node.js / npm	Node 18+、npm 9+	frontend 开发与构建
Docker Engine	24+	Compose 与镜像构建
Docker Compose	v2	本地全栈编排
kubectl	支持目标集群版本	Kustomize 渲染与 Kubernetes 部署

资源需求取决于 Docker Desktop、镜像缓存和集群实现，仓库未提供经实验确认的最低硬件门槛。课堂演示前应在实际设备上完成一次全栈预演，不应把建议配置写成已验证最低配置。

3.2 获取代码与配置

```
git clone https://github.com/daihao007/Lumalife.git
cd Lumalife
git status --short
git log -1 --oneline
```

创建本地环境文件：

```
cp .env.example .env
```

必须替换 `.env` 中的示例数据库密码；不得提交 `.env`、真实 API Key、Kubeconfig 或集群凭据。`AGNES_API_KEY` 可为空，此时 `assistant-service` 使用确定性降级回复。

4 Docker Compose 部署

4.1 默认全栈启动

```
docker compose config --services
docker compose up --detach --build --wait
docker compose ps
```

默认应包含：

```
mysql
rabbitmq
backend
frontend
identity-service
merchant-service
order-service
assistant-service
```

首次空数据卷启动时，MySQL 初始化脚本自动创建 legacy 与 3 个服务数据库并应用基线迁移。应用通过 `healthcheck` 和 `depends_on: condition: service_healthy` 控制启动顺序。

4.2 健康与访问入口

```
curl --fail http://localhost:8080/actuator/health
curl --fail http://localhost:5173/actuator/health
curl --fail http://localhost:8081/actuator/health
curl --fail http://localhost:8082/actuator/health
curl --fail http://localhost:8083/actuator/health
curl --fail http://localhost:8084/actuator/health
```

浏览器入口为 <http://localhost:5173/>。对外业务请求应通过 frontend 代理或 backend 8080 进入，不应让浏览器直接依赖内部服务 API。

4.3 数据生命周期工具

以下工具属于 db-tools profile，不随默认启动运行：

```
docker compose --profile db-tools run --rm db-migrate
docker compose --profile db-tools run --rm db-verify
docker compose --profile db-tools run --rm db-backfill-services
```

仅在本地演示或隔离测试环境写入样例数据：

```
docker compose --profile db-tools run --rm db-seed
```

清理业务数据会造成数据丢失，只能在确认目标为可销毁环境后执行：

```
docker compose --profile db-tools run --rm db-clean
```

4.4 停止与重建

保留数据卷停止：

```
docker compose down
```

`docker compose down --volumes --remove-orphans` 会删除 Compose 数据卷，只适用于明确授权的可销毁环境。生产或共享环境不得执行。

5 本地源码开发

本地直接运行适合开发调试，不等于默认微服务生产形态。

```
cd backend
mvn spring-boot:run

cd frontend
npm ci
npm run dev
```

frontend 的 Vite 代理默认指向 <http://localhost:8080>，可通过 `VITE_API_TARGET` 覆盖。若仅启动 backend 而未启动 4 个微服务，应明确选择 `monolith` 开发基线；不能把该模式描述为当前 Compose/Kubernetes 的默认链路。

6 Kubernetes 与 Kustomize

6.1 静态资源口径

`kubectl kustomize k8s` 的既有本地验证结果为 PASS，静态清单共 27 个对象：

资源	数量
Namespace	1
Deployment	7
StatefulSet	4

资源	数量
Service	11
HPA	2
PVC	2
Ingress	0

部署脚本运行时另创建 2 个 Secret 和 2 个 ConfigMap，因此它们不计入静态 27。静态最低 Pod 基线为 12；HPA 只覆盖 backend 与 merchant-service，目标 CPU 为 60%，min 1、max 3。

6.2 部署前检查

```
kubectl cluster-info
kubectl kustomize k8s > lumalife-k8s.yaml
```

生产部署必须使用不可变镜像标签，例如 `sha-<7位提交号>`。`scripts/deploy-k8s.sh` 会拒绝渲染结果中残留的 LumaLife :main 镜像或 `value: main` 服务版本。

所需敏感变量必须来自 Secret manager 或 CI secrets：

- LUMALIFE_INTERNAL_SERVICE_TOKEN
- RABBITMQ_USER
- RABBITMQ_PASSWORD
- MYSQL_PASSWORD
- MYSQL_ROOT_PASSWORD

6.3 版本化部署

```
export LUMALIFE_INTERNAL_SERVICE_TOKEN='<secret>'
export RABBITMQ_USER='<secret>'
export RABBITMQ_PASSWORD='<secret>'
export MYSQL_PASSWORD='<secret>'
export MYSQL_ROOT_PASSWORD='<secret>'
bash scripts/deploy-k8s.sh 'sha-abcdef0'
```

上述值均为占位符，不能复制为生产凭据。部署脚本执行镜像预拉取、Secret/ConfigMap 创建、legacy 数据迁移保护、服务数据库回填、版本化清单应用、rollout 等待、探针验证和集群内健康检查；失败时输出工作负载、Pod、日志与事件诊断。

6.4 探针与服务暴露

所有应用工作负载均配置 readiness、liveness 和 startup probes，并声明资源 requests/limits。仓库静态清单没有 Ingress；外部访问方式由目标环境另行配置，不能假设已经存在域名、TLS 或公网入口。

7 CI/CD 部署链

`.github/workflows/ci.yml` 的逻辑顺序为：

- 11 类验证
- > quality-gate
- > 6 个应用镜像
- > 一次性 Kind smoke
- > 一次性 Kind acceptance deploy

main push 成功时发布 `sha-<短提交号>` 与 main 标签，但 Kubernetes 部署消费不可变 `sha-*` 标签。ECS workflow 只在 main push 的主 CI 成功、ECS 开关启用且自托管 Runner 就绪时下载同一 run 的部署 bundle，并把同一提交版本部署到 K3s。

当前 HEAD 尚未取得完整 GitHub Actions 成功证据，因此只能说明配置具备上述失败阻断关系，不能宣称当前提交已自动部署成功。

8 验证与证据状态

验证项	当前证据	状态
Compose 配置解析	前序本地 <code>docker compose config</code>	LOCAL-VERIFIED
默认服务数	8	LOCAL-VERIFIED
Kustomize 渲染	27 个静态对象	LOCAL-VERIFIED
部署脚本静态门禁	<code>scripts/test-deploy-k8s.sh</code>	前序 PASS
当前 HEAD 完整 CI	无新 run	NOT-RUN
当前 HEAD ECS/K3s 部署	无新 artifact	UNVERIFIED
HPA 实验	8c335eb, 1->2->3->1	EXISTING-EVIDENCE-VERIFIED

本次未运行上述命令或修改服务器；前序服务器旧提交不能作为当前 HEAD 部署证据。

9 回滚与故障处理

9.1 应用回滚

应用回滚应部署已验证的旧 `sha-*` 镜像，而不是可变 `main`；回滚前后记录提交、镜像标签、rollout 和健康结果。

9.2 数据回滚边界

数据库迁移和业务数据可能不可逆。脚本的 legacy 备份与服务库回填不等于通用数据库回滚；恢复前应停止写入、保存备份并人工评估。

9.3 常见排障

现象	优先检查
Compose 服务不健康	<code>docker compose ps</code> 、对应服务日志、MySQL/RabbitMQ healthcheck
backend 返回 503	4 个下游服务健康、服务 URL、内部 Token；这不是 circuit breaker
登录或数据为空	profile、服务数据库名、迁移/seed 是否按目标环境执行
Kubernetes rollout 超时	<code>kubectl get pods</code> 、 <code>describe</code> 、当前/previous logs、events
ImagePullBackOff	镜像标签、GHCN 权限、节点架构、不可变 tag 是否存在
HPA 不变化	metrics-server、CPU requests、负载持续时间和 HPA target

同步 HTTP 失败处理是 timeout、显式 503 与隔离；没有通用 circuit breaker、rate limit 或同步 retry。

10 部署结论

当前仓库具备 8 服务 Compose、版本化迁移、27 个静态 Kubernetes 对象、6 个应用镜像、探针、HPA 和质量门部署链；当前 HEAD 的完整 CI 与 ECS/K3s 部署尚未重验。

部署交付状态为 PARTIAL。答辩前仍应在实际设备或隔离环境执行提交绑定的全栈启动与健康检查并保存输出；恢复测试授权前，本报告不把这些步骤标为已执行。